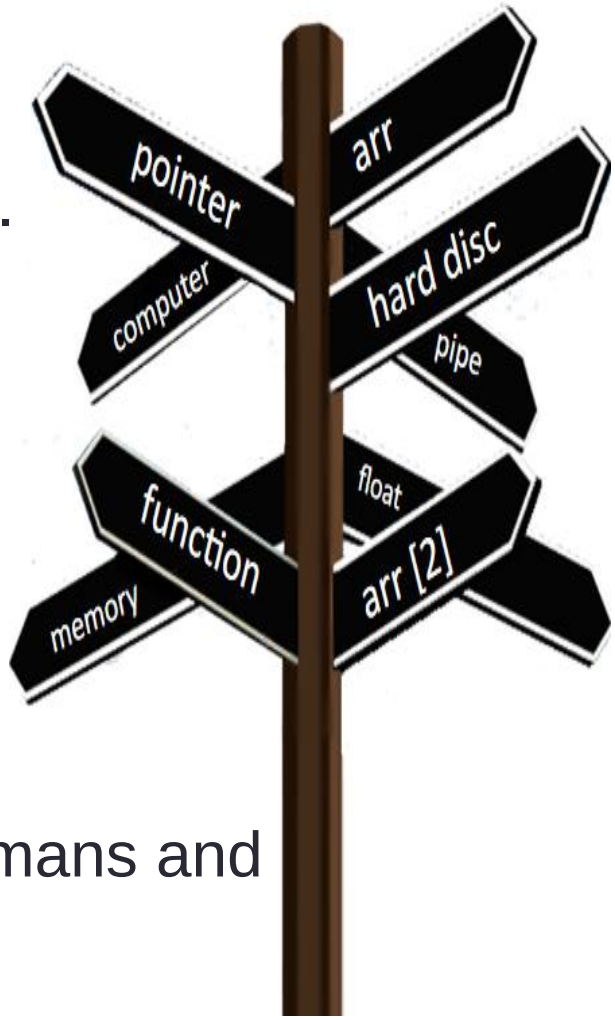
POINTERS

CS217 – Object Oriented Programming

Pointers – the absolute basics

- **Pointers** are memory locations used to save values in C++.
- Pointers are different from other types.
  - The types we've been using are closely linked to computer data processing and reflect our ideas and intuition.
    - Integers are used to count.
    - *floats* used when pay for something.



- **Pointers** have no simple and obvious analogy to our daily lives and their values are unreadable to humans and completely useless.

Pointers – the absolute basics

- Each declared variable occupies a small piece of the computer memory. The attribute of where this value is stored is called the **address**.

- Difference between variable content and address:
  - The **value** of the variable is **what** the variable stores;
  - The **address** of the variable is information about **where** this variable is placed.

- **Pointers are used to store information about the location (address) of any other data**.

Pointers Declaration: The first pointer

- To declare an integer pointer we use the following syntax:

```
int *p;
```

- This declaration sets up a variable named *p*. It isn't an *int* - **the asterisk (*) means that *p* is a pointer -** and will be used to store information about the location of the data of type *int*.
- The pointers are always used to point to the specific data referred to in the declaration.

How to assign a value

- *How do we assign a value to the* ***pointer variable****?*
  - by using the = operator.
- **Using a literal is not an option;** the “C++” language syntax doesn’t allow it.

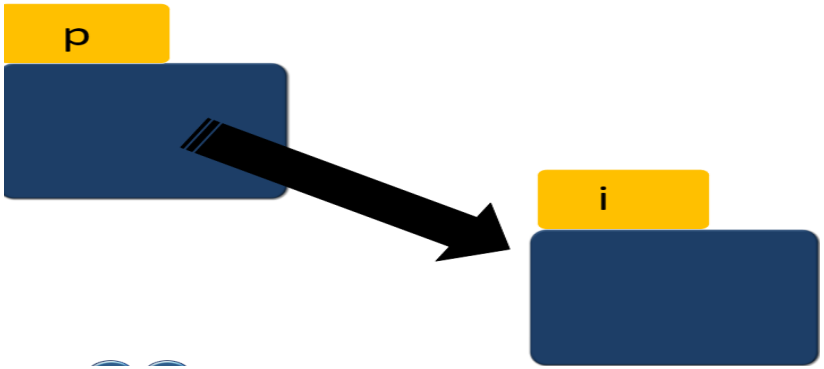
~~p = 148324;~~

- There’s one distinctive exception. **zero can be assigned to the pointer variable**.
- p = 0; OR p = NULL;
- A pointer that is assigned a value of zero is called **a null pointer**.
- A **null pointer** doesn’t point to anything.

How to assign a value

- Pointers can be assigned the value which **points to any already existing variable**.
- To do that, we need an & operator called the **reference operator**.
  - The & operator is a unary prefix operator
  - The operator simply finds the address of its argument.
  - After completing the assignment, the *p* variable will point to the place where the *i* variable is stored in the memory.

```
p = &i;
```



How to assign a value

- *How do we get a value pointed to by the pointer?*
- Answer: We can **dereference** it.
- **Dereferencing is an operation where the pointer variable becomes synonymous with the value it points to**.
- To declare a variable of type *int*(*ivar*) and a variable of type *int* * (*ptr*). We can do it within a single statement

```
int ivar, *ptr;
```

How to assign a value

- To assign the value of 2 to the *ivar* variable.

```
ivar = 2;
```

- Now we make the *ptr* pointer point to the *ivar* variable.

```
ptr = &ivar;
```

- To get the value pointed to by the pointer we have to use the operator (an asterisk: "*") but in a completely new situation – as a **dereferencer**.
  - If you put an asterisk in front of a pointer, you get a value which is stored at the location pointed to by the pointer.

```
*ptr
```

- **Dereferencing NULL pointers is forbidden** and leads to serious problems very quickly.

How to assign a value

- *To print the value pointed to by the pointer*

```
cout << *ptr;
```

- *To set the value pointed to by the pointer.*

```
*ptr = 4
```

  - This statement **won't change the pointer value**; it will change the value pointed to by the pointer instead.

sizeof operator

- The ***sizeof*** operator is **a unary prefix operator** with the highest possible priority.
- The ***sizeof*** operator expects its argument to be a literal or a variable or an expression enclosed in parentheses or the type name.
  - It is the only “C” operator that allows its argument to be a type.
- The ***sizeof*** operator provides information on **how many bytes of memory its argument occupies** (or may occupy).
- **The *sizeof* is not only an operator – it’s also a keyword.**

sizeof

sizeof operator

- Example 1:

```
int i; char c;
i = sizeof c;
```

- Variable *i* will be assigned the value of 1 because the char values occupy one byte.
- Same effect can be achieved by writing:

  i = **sizeof**(char);

  - The parentheses may not be used when the argument is a literal or a value, but you do have to use the parentheses when the argument is a type.

sizeof operator

- Example 2:

```
char tab[10];
i = sizeof tab;
```

  - Variable *i* will be set to the value of 10 because this is the number of bytes occupied by the **entire** *tab*array.

- Example 3:

```
char tab[10];
i = sizeof tab[1];
```

  - Variable *i* will be set to the value of 1

sizeof operator

- Example 4: check this out!

```
#include<iostream>
using namespace std;

int main()
{
        char *p; //pointer to char variable
        char i;  // var of type char
        p=&i;
        cout<<sizeof(i)<<"    "<<sizeof(p)<<endl;
        return 0;
}
```

The output: 1 4
why?

Pointers vs. arrays

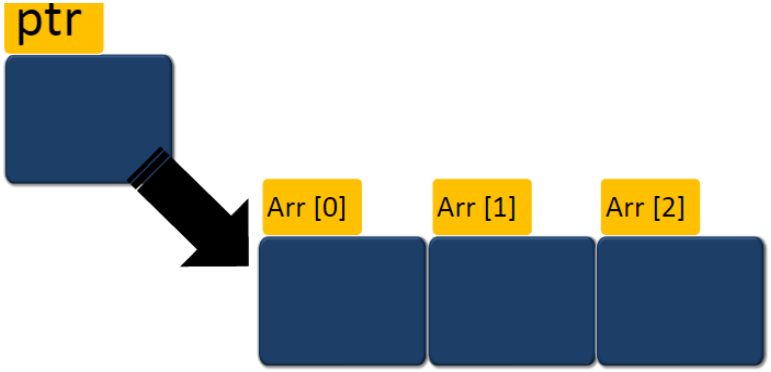
- The name of an array without indices, it's a synonym of the pointer pointing to the first element of the array.
- Example:

```
int *Ptr, Arr[3];
```

  - A pointer to *int* and the three-element array of type *int are* declared.

```
Ptr = &Arr[0];
Ptr = Arr;
```

  - These two assignments will set *Ptr* to the same value.
    In other words, the following comparison is **always true**:
    Arr == &Arr[0]



The pointers' arithmetic

- The pointers' arithmetic is different from the integers' arithmetic as it is relatively reduced and allows the following operations only:
  - **Adding an integer** value to a pointer giving a pointer (*ptr* + *int* → *ptr*)
  - **Subtracting an integer** value from a pointer giving a pointer (*ptr* – *int* → *ptr*)
  - **Subtracting a pointer from a pointer** giving an integer (*ptr* – *ptr* → *int*)
  - **Comparing the two pointers** for equality or inequality (such a comparison gives a value of type *int* representing true or false) (*ptr* == *ptr*→ *int*; *ptr* != *ptr* → *int*)
- **Any other operations** are either **prohibited** or meaningless. These are the only operations you can use.

The pointers' arithmetic

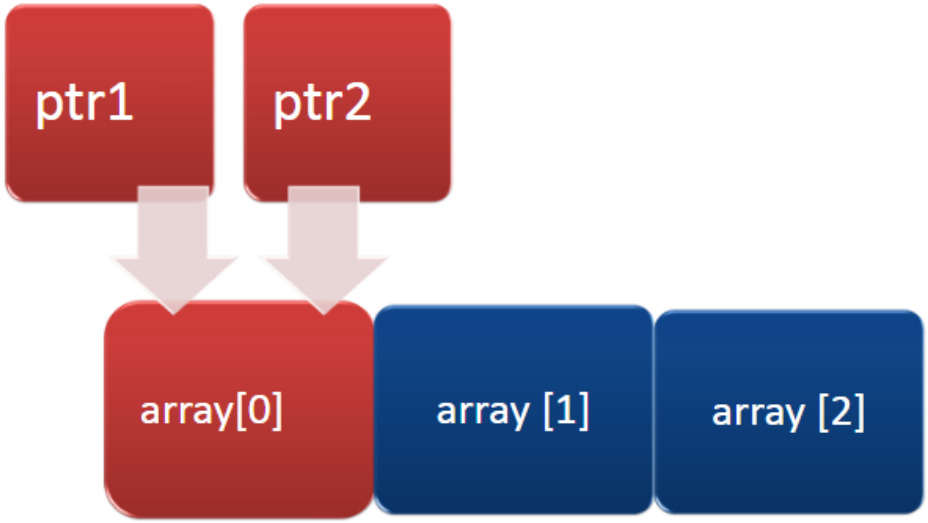
- Example:

```
int *ptr1, *ptr2, array[3], i;

ptr1 = array;
```

  - *ptr1* points to the first element of the *array*.
  - After the following assignment, ***ptr2* points to the first element of the *array***

```
ptr2 = ptr1;
```



The pointers' arithmetic

- To check if the two pointers are equal equality operator is used.

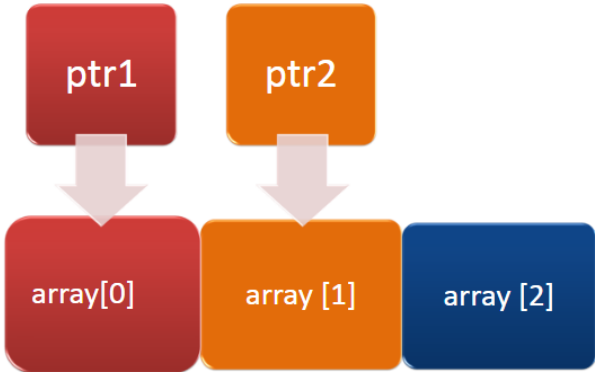
```
if(ptr2 == ptr1) {
:
:
}
```

  - Two pointers are equal if they point to the same memory location or the same element of an *array*.

The pointers' arithmetic

- To check how addition works. These statements perform the same operation: they add 1 to *ptr2*.
  - The interpretation of this operation is as follows:

```
ptr2 = ptr2 + 1;
ptr2++;
```



- it’s taken into account what type is pointed to by the pointer - in our example it’s *int*

The pointers' arithmetic

- The addition determines **how many bytes of memory the type occupies** .
  - The value we want to add to the pointer is multiplied by the size of the pointer type.
  - In effect, the pointer moves itself to the next *int* value in the memory.
- What would happen if we added 2 instead of 1 to *ptr2* pointer?
  - In this case, the *ptr2* would increase by (2 * *4)*
  - and *ptr2* would move through **two** *int* values and point to the third element of the *array*(namely, *array[2]*).
  - The comparison: ptr1 == ptr2 is obviously false.
  - The comparison: ptr1 != ptr2 is true, as both pointers **point to completely different data**.

The pointers' arithmetic

- The **pointers subtraction gives a result of type *int***. Both pointers need to **point to the same type**; the compiler will check it.

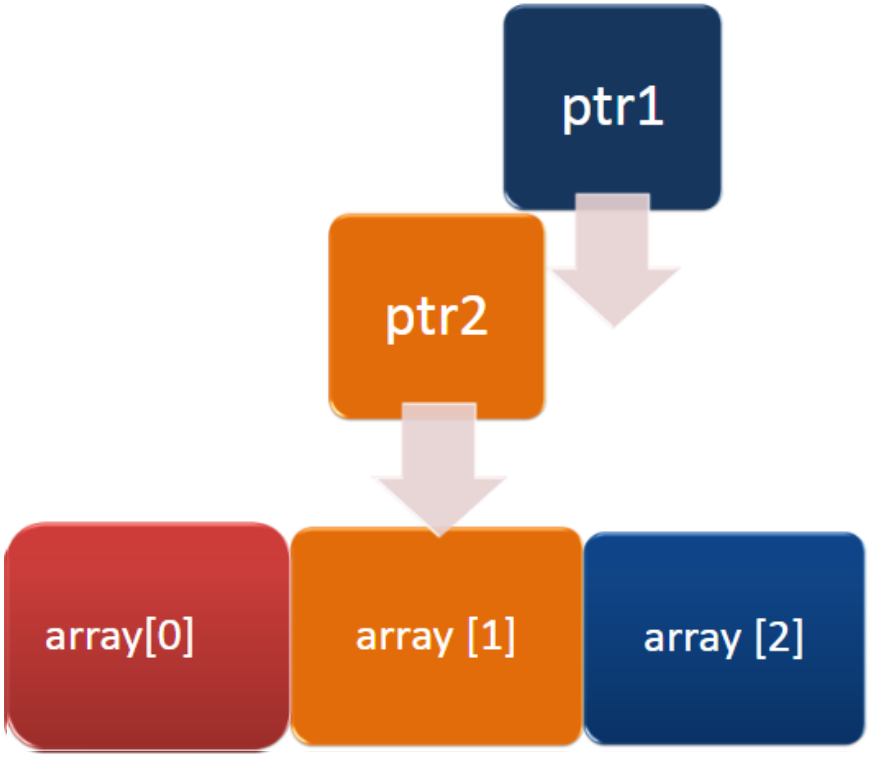
```
i = ptr2 - ptr1;
```

  - The final result tells us how many variables of a given type (i.e. *int*) fit between the addresses stored in the pointers. In our case it's 1, and this value will be assigned to the *i* variable.
  - The result will be greater than 0 if *ptr2* points to the memory located after *ptr1*; otherwise, it'll be less than 0.

The pointers' arithmetic

- What will be the result of the following operation

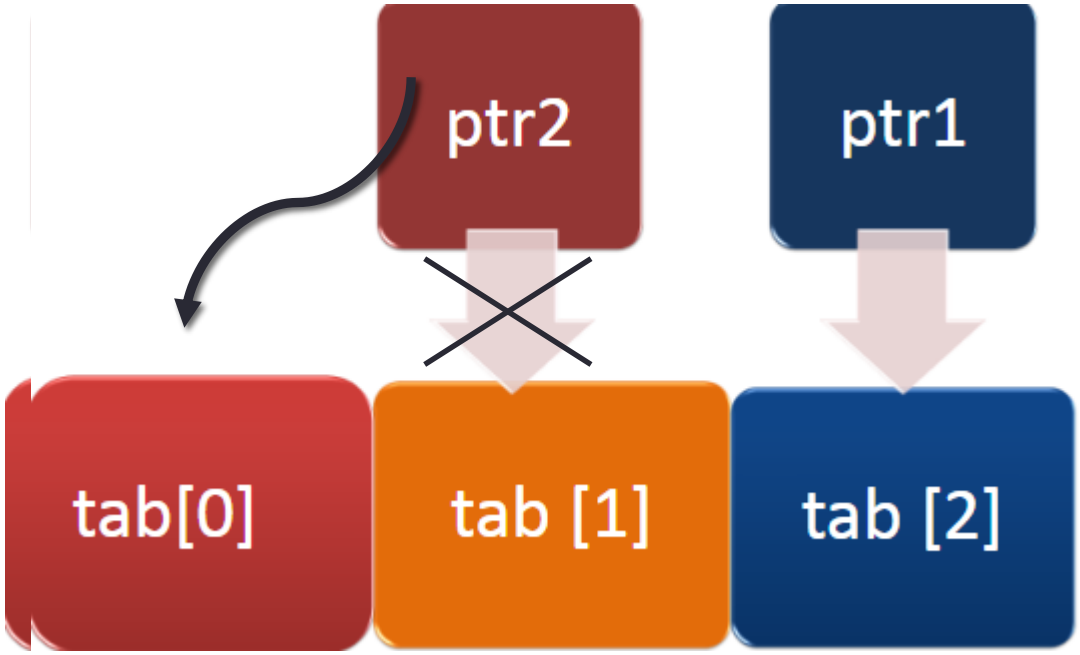
```
ptr1 = ptr1 + 2;
```



The pointers' arithmetic

- Example: Assume the following operation

```
ptr2 = ptr2 – 1;
```



The pointers' arithmetic

- Example: What will be the result of the following operation.

ptr1 - ptr2

?

- Is it 2?

Memory on demand

- In our examples so far, all work associated with memory allocation was managed by the compiler.
- High level languages are designed to spare the developers from such responsibilities.
- In some cases, developer wants to have full control over how much memory is used and when exactly it's used.
  - When you don't know in advance what the size of the data to be processed is.
- To manage memory (allocating and freeing), the "C++" language gives us **two** specialized keywords.

1. **`new`** : used **to request the creation of a new memory block.**
2. **`delete`** : used to release (free) allocated data and return it to the management of the operating system.

new

- The **new** keyword can be used in the following way:

```
float *array = new float[20];
int  *count = new int;
```

- the **new** **returns a pointe**r of type conforming to the newly created entity
- the newly allocated memory area **is not filled (initiated)** in any way, so you should expect it to just contain **garbage.**

delete

- When we no longer need the memory, we can release it (free it) using the **delete** keyword in the following way:

```
delete [] array;
delete count;
```

- we use the **delete[]** form if we want to free up the memory allocated for an array, otherwise we use **delete**.
  - you can only release the entire allocated block, not a part of it.
- After performing **delete**, all the pointers that point to the data inside the freed area become illegal;
  - attempting to use them may result in an abnormal program termination.

Memory on demand – example 1

- We declare a variable called **`arr`** which will point to the data of type **`float`** (the pointer's type is **`float *`**); no value is initially assigned to this variable.
- We use the **`new`** keyword to allocate a block of memory sufficient to store a float array consisting of 5 elements.
- We make use of the newly allocated array, then we release it using the **`delete`** keyword.
- Pay attention to the fact that the pointer returned by **`new`** is treated as if it’s an array!
- The handling of the ***dynamic arrays*** (created during the run of the program) is no different than using regular arrays declared in the usual way.
- Regardless of the nature of the array, we can access its elements using **`[]`** operator (the same old way).

```
1   #include<iostream>
2
3   using namespace std;
4
5   int main()
6   {
7       float *arr;
8       arr = new float[5];
9       for (int i = 0; i < 5; i++)
10          arr[i] = i*i;
11
12      for (int i = 0; i < 5; i++)
13          cout<<arr[i]<<endl;
14
15      delete[] arr;
16
17      return 0;
18  }
```

Memory on demand

- In the static array, the size must be constant because the allocation is done in the compile time. So, the following code will cause an error:

```
int x;
cin>>x;
int arr[x];
```

- In the dynamic array, the size can be variable because the allocation is done in the run-time.

```
int x;
cin>>x;
int *arr = new int [x];
```